

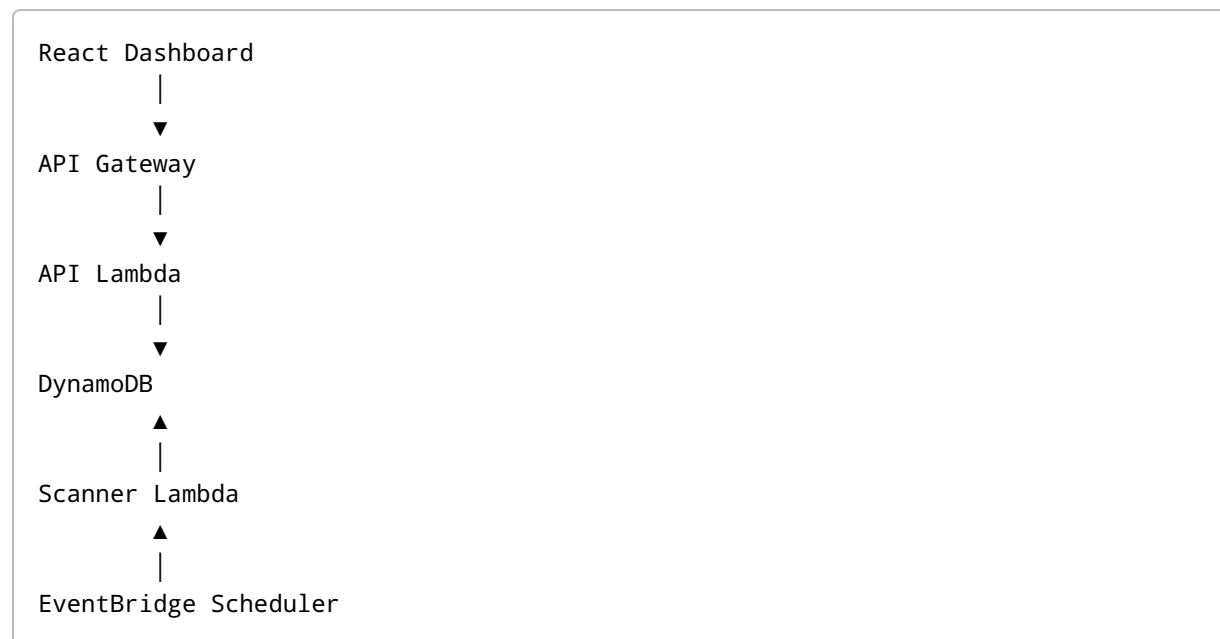
AWS Cloud Security Dashboard - Architecture Documentation

1. Overview

The AWS Cloud Security Dashboard is a serverless cloud security monitoring solution designed to continuously scan AWS resources, identify security misconfigurations, persist findings, and visualize security posture through a modern web dashboard.

The solution leverages AWS serverless services and Infrastructure as Code (IaC) principles using Terraform.

2. High-Level Architecture



3. Component Responsibilities

Frontend (React + TypeScript)

Responsibilities:

- Display security findings.
- Visualize metrics using charts.
- Show historical findings.

- Track lifecycle of findings.
- Communicate with backend APIs.

Technology:

- React
 - TypeScript
 - Material UI
 - Axios
 - Vite
-

API Gateway

Responsibilities:

- Expose REST endpoints.
- Route incoming requests to Lambda.
- Provide public API access.

Endpoints:

- GET /summary
 - GET /findings
 - GET /history
 - GET /lifecycle
-

API Lambda

Responsibilities:

- Process API requests.
- Query DynamoDB.
- Aggregate security metrics.
- Build lifecycle information.

Services:

- findings_query_service
 - summary_service
 - lifecycle_service
-

Scanner Lambda

Responsibilities:

- Perform AWS security scans.
- Identify security findings.
- Persist findings.
- Trigger SNS alerts.

Security Checks:

- Security Group exposure.
 - S3 encryption.
 - S3 versioning.
 - IAM AdministratorAccess assignments.
-

DynamoDB

Responsibilities:

- Persist all historical findings.
- Provide data source for dashboard analytics.

Characteristics:

- Fully managed.
 - Highly available.
 - Serverless.
-

EventBridge

Responsibilities:

- Schedule automated security scans.

Schedule:

```
rate(1 day)
```

SNS

Responsibilities:

- Notify administrators of critical findings.
-

4. Request Flow

Dashboard Request

```
graph TD; User --> ReactDashboard[React Dashboard]; ReactDashboard --> APIGateway[API Gateway]; APIGateway --> APILambda[API Lambda]; APILambda --> DynamoDB; DynamoDB --> JSONResponse[JSON Response]; JSONResponse --> DashboardRendering[Dashboard Rendering];
```

5. Security Considerations

Implemented:

- IAM least privilege permissions.
 - Infrastructure provisioning through Terraform.
 - No hardcoded AWS credentials.
 - Environment variables for configuration.
 - Public frontend isolated from backend logic.
-

6. Terraform Module Structure

Modules:

- api_gateway
- api_lambda
- dynamodb

- eventbridge
- frontend_hosting
- iam
- lambda
- sns

Benefits:

- Reusability
- Maintainability
- Separation of concerns
- Environment portability

7. Future Improvements

Potential enhancements:

- CIS Benchmark support
- AWS Config integration
- CloudTrail analysis
- MFA compliance checks
- Multi-account scanning
- Authentication using Cognito
- CloudFront CDN integration